
Ampyr-PSP Technical Reference

Project Sizing Platform -- BESS & DG Sizing Tool

Version	1.2.0
Author	Ankit Agarwal, GM -- Product & Technology, Ampyr GTC
Audience	DoublU Development Team (React + FastAPI rebuild)
Last Updated	2026-03-17
Prototype Stack	Python 3.11+, Streamlit 1.41, Pandas 2.2, NumPy 2.1, Plotly 5.24
Production Stack	React + TypeScript, MUI, FastAPI, PostgreSQL, Docker

CONFIDENTIAL -- AMPYR Energy / DoublU India Pvt Ltd

Table of Contents

1. [Executive Summary](#)
2. [Architecture Overview](#)
3. [Step 1 — System Setup](#)
4. [Step 2 — Dispatch Rules](#)
5. [Step 3 — BESS & DG Sizing](#)
6. [Core Simulation Engine](#)
7. [Step 4 — Results & Analysis](#)
8. [Step 5 — Multi-Year Projection](#)
9. [Degradation Engine](#)
10. [Fuel Model](#)
11. [Step 6 — Green Energy Analysis](#)
12. [Step 7 — Financial Analysis](#)
13. [Market Reference Page](#)
14. [Configuration & Defaults Reference](#)
15. [Appendices](#)

1. Executive Summary

What the Tool Does

The **Project Sizing Platform (PSP)** is a decision-support tool for sizing Battery Energy Storage Systems (BESS) in Solar + Storage + Diesel Generator (DG) hybrid projects. Given a load demand profile and solar generation data, it:

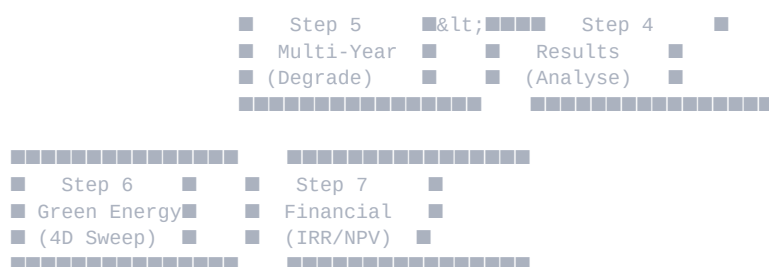
1. Simulates **8,760 hours** (1 year) of battery operations with configurable dispatch rules
2. Tests hundreds of BESS + DG configurations in a parametric sweep
3. Ranks optimal configurations by delivery performance, green energy %, wastage, and cost
4. Projects performance over **10–20 years** with battery degradation
5. Performs a **4D green energy optimisation** sweep (Solar x BESS x Container x DG)
6. Calculates **ungeared Project IRR** using a 35-year monthly FCFF financial model

User Journey (7-Step Wizard)

Step 1: Setup	→ Define load, solar, BESS, DG, degradation parameters
Step 2: Dispatch Rules	→ Configure DG timing, triggers, priorities → auto-infer template
Step 3: Sizing	→ Set capacity ranges, run parametric sweep simulation
Step 4: Results	→ View, filter, compare, and export simulation results
Step 5: Multi-Year	→ Project degradation and performance over 10-20 years
Step 6: Green Energy	→ 4D optimisation sweep for green energy targets
Step 7: Financial	→ Input costs/revenue, calculate Project IRR (XIRR)

Key Business Constraints

- **Binary Delivery:** Deliver the full load MW target each hour or deliver nothing (no partial delivery)
- **Solar-Only Charging:** Battery charges from solar (or DG excess if configured), never from grid
- **Cycle Limits:** Max 2.0 equivalent cycles per day (configurable, hard enforcement)
- **SOC Bounds:** Battery SOC must stay within [min_soc, max_soc] at all times



State Management

All configuration persists in `st.session_state.wizard_state` — a nested dictionary with sections:

- `setup` — Step 1 parameters
- `rules` — Step 2 dispatch rules
- `sizing` — Step 3 range definitions
- `results` — Step 4 cached simulation results
- `financial` — Step 7 financial parameters

Key functions:

- `init_wizard_state()` — Initialise defaults if not present
- `get_wizard_state()` — Read current state
- `update_wizard_state(section, key, value)` — Write to specific section
- `validate_step_N()` — Returns `(is_valid, errors_list)` for step N

3. Step 1 — System Setup

File: `pages/Step1_Setup.py` (903 lines) **Purpose:** Configure load demand, solar generation, BESS parameters, DG options, and degradation strategy.

3.1 Load Profile

Defines the 8,760-hour electricity demand pattern the system must serve.

Mode Selection (`setup.load_mode`):

Mode	Description	Parameters
constant	Fixed MW every hour	<code>load_mw</code>
day_only	MW during day hours, 0 at night	<code>load_mw</code> , <code>load_day_start</code> , <code>load_day_end</code>
night_only	MW during night hours, 0 during day	<code>load_mw</code> , <code>load_night_start</code> , <code>load_night_end</code>
seasonal	MW during specific months + daily window	<code>load_mw</code> , <code>load_season_start/end</code> , <code>load_season_day_start/end</code>
custom	Multiple user-defined time windows	<code>load_windows[]</code> (each: start, end, mw)
csv	User-uploaded 8,760-hour CSV	<code>load_csv_data</code>

Built-in Presets:

- `constant_25mw` — 25 MW, all hours
- `constant_50mw` — 50 MW, all hours
- `office_hours` — 25 MW, hours 8–18
- `evening_peak` — 25 MW, hours 17–23
- `night_operations` — 25 MW, hours 18–6 (crosses midnight)
- `two_shift` — 25 MW, hours 6–14 and 14–22

Load Builder Formulas (`src/load_builder.py`):

```
# Constant mode
load_profile = np.full(8760, load_mw)

# Day only
for hour in range(8760):
    hod = hour % 24
    if day_start <= hod < day_end:
        load_profile[hour] = load_mw

# Night only (handles midnight wraparound)
for hour in range(8760):
    hod = hour % 24
    if night_start > night_end: # e.g., 18 to 6
        if hod >= night_start or hod < night_end:
            load_profile[hour] = load_mw
    else:
        if night_start <= hod < night_end:
            load_profile[hour] = load_mw

# Seasonal
for hour in range(8760):
    day_of_year = hour // 24 + 1
    hod = hour % 24
    if _is_in_month_range(day_of_year, start_month, end_month):
        if _is_in_range(hod, day_start, day_end):
            load_profile[hour] = load_mw
```

Load Analysis Metrics:

```
total_energy_mwh = sum(load_profile)
peak_load_mw = max(load_profile)
load_hours = count(load_profile > 0)
load_factor = total_energy_mwh / (peak_load_mw * 8760)
```

Input Fields:

Field	Type	Min	Max	Default	Step
load_mw	number_input	1.0	500.0	25.0	5.0
load_day_start	slider	0	23	6	1
load_day_end	slider	0	23	18	1
load_night_start	slider	0	23	18	1
load_night_end	slider	0	23	6	1
Season months	selectbox	1	12	4/10	—
Custom window MW	number_input	1.0	500.0	25.0	5.0

3.2 Solar Profile

Defines the hourly solar generation used as the primary energy source.

Sources:

- **Inputs folder:** Pre-loaded CSV files from Inputs/ directory (default: Solar Profile.csv, 67.9 MW peak)
- **Upload:** User-uploaded CSV file

Validation Requirements:

- Must contain exactly 8,760 rows (one per hour of year)
- All values must be ≥ 0 (no negative generation)
- Values should not exceed rated capacity $\times 1.1$

Scaling Formula (src/data_loader.py):

```
scaled_profile = base_profile * (target_capacity / base_peak_capacity)
```

Solar Statistics:

```
max_mw = max(profile)
mean_mw = mean(profile)
total_mwh = sum(profile)
capacity_factor = total_mwh / (max_mw * 8760) * 100
zero_hours = count(profile == 0)
```

3.3 BESS Parameters

Container Types (multiselect):

Container	Energy	Power	C-Rate
5mwh_2.5mw	5 MWh	2.5 MW	0.5C (2-hour duration)
5mwh_1.25mw	5 MWh	1.25 MW	0.25C (4-hour duration)

Input Fields:

Field	Type	Min	Max	Default	Step	Unit
bess_efficiency	slider	70	95	87	1	% (round-trip)
bess_min_soc	slider	0	50	5	5	%
bess_max_soc	slider	50	100	95	5	%
bess_initial_soc	slider	min_soc	max_soc	50	5	%
bess_daily_cycle_limit	number_input	0.5	3.0	2.0	0.1	cycles/day
bess_enforce_cycle_limit	checkbox	—	—	True	—	boolean

Derived Constants (computed at simulation initialisation):

```

one_way_efficiency = sqrt(bess_efficiency / 100)    # e.g., sqrt(0.87) = 0.933
usable_capacity = capacity * (max_soc - min_soc) / 100
min_soc_mwh = capacity * min_soc / 100
max_soc_mwh = capacity * max_soc / 100
charge_power_limit = capacity * C_rate_charge      # MW
discharge_power_limit = capacity * C_rate_discharge # MW

```

3.4 Diesel Generator (DG)

Field	Type	Min	Max	Default	Step	Unit
dg_enabled	checkbox	—	—	True	—	boolean
dg_operating_mode	radio	—	—	binary	—	binary / variable
dg_min_load_pct	slider	10	50	30	5	% (variable mode only)

Fuel Model Parameters (when dg_fuel_curve_enabled = True):

Field	Type	Min	Max	Default	Step	Unit
dg_fuel_f0	number_input	0.01	0.10	0.03	0.005	L/hr/kW (no-load)
dg_fuel_f1	number_input	0.15	0.35	0.22	0.01	L/kWh (load)
dg_fuel_flow_rate	number_input	0.15	0.40	0.25	0.01	L/kWh (simple model)
dg_fuel_price	number_input	0.50	5.00	1.50	0.10	\$/L

3.5 Degradation Strategy

Strategy	Description
standard	Battery operates at nameplate capacity, degrades naturally
overbuild	Install 20% extra capacity to offset future degradation
augmentation	Add replacement capacity at year 8

Calendar Degradation: 2%/year baseline **Cycle Degradation:** 0.15% per equivalent full cycle

3.6 Validation Rules

- Solar profile must have 8,760 values
- min_soc < max_soc
- initial_soc within [min_soc, max_soc]
- At least one container type selected
- Load profile must be buildable from selected mode/parameters

4. Step 2 — Dispatch Rules

File: pages/Step2_Rules.py (443 lines) **Purpose:** Configure how the DG interacts with load, solar, and BESS. Auto-infers the dispatch template (T0–T6).

> **Note:** If DG is disabled in Step 1, this page shows "No DG configured" and template T0 is assigned automatically.

4.1 Six Configuration Questions

Q1: DG Timing — "When can the generator run?"

Option	Value	Description
Anytime	anytime	No time restriction
Day only	day_only	Only during configurable day hours
Night only	night_only	Only during configurable night hours
Custom blackout	custom_blackout	Cannot run during specified hours

Additional inputs per option:

- day_only: day_start_slider (0–23), day_end_slider (0–23)
- night_only: night_start_slider (0–23), night_end_slider (0–23)
- custom_blackout: blackout_start_slider (0–23), blackout_end_slider (0–23)

Q2: DG Trigger — "What triggers the generator?"

Available options depend on timing selection:

Timing	Available Triggers
anytime	reactive (load deficit), soc_based (SOC threshold)
day_only	soc_based only
night_only	proactive (pre-emptive charging), soc_based
custom_blackout	reactive only

SoC-based trigger additional inputs:

- soc_on_threshold: slider, min=bess_min_soc, max=bess_max_soc–10, step=5 (default 30%)
- soc_off_threshold: slider, min=soc_on+10, max=bess_max_soc, step=5 (default 80%)

Q3: DG Charges BESS

Option	Value	Description
No — solar only	False	Excess DG output is curtailed
Yes — excess charges BESS	True	DG excess power is stored in battery

Q4: Load Serving Priority

Option	Value	Description
BESS first	bess_first	Discharge battery before starting DG
DG first	dg_first	Start DG before using battery

Q5: DG Takeover Mode

Option	Value	Description
No — DG fills gap	False	DG supplements solar+BESS shortfall
Yes — DG serves full load	True	When activated, DG serves entire load; solar diverted to BESS

Q6: Cycle Charging (only when DG mode = variable)

Field	Type	Min	Max	Default	Step
Enabled	radio	—	—	False	—
Min DG load	slider	50	90	70	5
Off SOC	slider	soc_on+20	max_soc	80	5

4.2 Template Inference

Module: src/template_inference.py

The system auto-maps user answers to a dispatch template ID:

Template	Name	Conditions	Merit Order
T0	Solar + BESS Only	DG disabled	Solar -> Battery -> Unserved
T1	Green Priority	Anytime + Reactive	Solar -> Battery -> DG -> Unserved
T2	DG Night Charge	Night + Proactive	Solar -> DG -> Battery
T3	DG Blackout Window	Custom blackout	Solar -> Battery -> DG (when allowed)
T4	DG Emergency Only	Anytime + SoC-based	Solar -> Battery -> DG (SoC trigger)
T5	DG Day Charge	Day only (always SoC)	Solar -> Battery -> DG (day only)
T6	DG Night SoC Trigger	Night + SoC-based	Solar -> Battery -> DG (night, SoC)

Inference Decision Tree:

```
if not dg_enabled → T0
elif dg_timing == 'anytime':
```

```

    if dg_trigger == 'reactive' → T1
    else → T4 (soc_based)
elif dg_timing == 'day_only' → T5
elif dg_timing == 'night_only':
    if dg_trigger == 'proactive' → T2
    else → T6 (soc_based)
elif dg_timing == 'custom_blackout' → T3

```

4.3 Validation

- SoC ON threshold must be less than OFF threshold
- Warning if deadband < 20% (may cause frequent DG cycling)
- Blackout start and end cannot be the same hour

5. Step 3 — BESS & DG Sizing

File: pages/Step3_Sizing.py (583 lines) **Purpose:** Define the range of BESS and DG configurations to test, then run the parametric sweep simulation.

5.1 Input Fields

BESS Capacity Range:

Field	Type	Min	Max	Default	Step	Unit
capacity_min	number_input	5	500	25	5	MWh
capacity_max	number_input	cap_min	1000	150	5	MWh

Duration Classes: Automatically derived from selected container types in Step 1:

- 5mwh_2.5mw → 2-hour duration
- 5mwh_1.25mw → 4-hour duration

DG Capacity Range (if DG enabled):

Field	Type	Min	Max	Default	Step	Unit
dg_min	number_input	0	200	load_mw	5	MW
dg_max	number_input	dg_min	200	load_mw	5	MW
dg_step	selectbox	—	—	5	—	MW

5.2 Configuration Count Formula

```

capacity_count = ((capacity_max - capacity_min) / 5) + 1
duration_count = len(selected_container_types) # 1 or 2
dg_count = ((dg_max - dg_min) / dg_step) + 1 # 1 if DG disabled

total_configurations = capacity_count × duration_count × dg_count

```

5.3 Optimisation Goals

Delivery Mode (delivery_mode):

Mode	Description
maximize	Find configs that maximise delivery %
at_least	Configs delivering $\geq$ target %
exactly	Configs delivering exactly target % ($\pm 0.5\%$)

Optimisation Priority (optimize_for):

Priority	Sorts by
min_bess_size	Smallest BESS capacity meeting goal
min_wastage	Lowest solar curtailment %
min_dg_hours	Fewest DG runtime hours
min_cycles	Fewest BESS equivalent cycles

Secondary Constraints:

- max_wastage_pct — Maximum acceptable solar curtailment %
- max_dg_hours — Maximum acceptable DG runtime hours

5.4 Simulation Execution

For each configuration in the sweep:

1. Compute BESS power from capacity and duration: $\text{power_mw} = \text{capacity_mwh} / \text{duration_hrs}$
2. Compute container count: $\text{containers} = \text{ceil}(\text{capacity_mwh} / \text{container_energy_mwh})$
3. Build simulationParams from wizard state + this configuration
4. Run `dispatch_engine.run_simulation(params)` → 8,760 hourly results
5. Calculate SummaryMetrics from hourly results
6. Store row in results DataFrame

Estimated Runtime: ~50ms per configuration

5.5 Results Columns

Each row in the sweep results contains:

Column	Unit	Description
BESS (MWh)	MWh	Battery energy capacity
Duration (hr)	hours	Discharge duration at rated power
Power (MW)	MW	Battery power rating
Containers	count	Number of physical containers
DG (MW)	MW	Diesel generator capacity
Delivery %	%	Hours with full load served / total load hours

Column	Unit	Description
Green %	%	Hours with load served without DG / total load hours
Wastage %	%	Solar energy curtailed / total solar generated
Delivery Hrs	hours	Count of hours with full delivery
Load Hrs	hours	Count of hours with load > 0
Green Hrs	hours	Count of hours with green-only delivery
DG Hrs	hours	DG runtime hours
DG Starts	count	Number of DG start events
BESS Cycles	cycles	Total equivalent full cycles
Unserved (MWh)	MWh	Total unmet energy demand
Fuel (L)	litres	Total DG fuel consumed

6. Core Simulation Engine

File: src/dispatch_engine.py (1,241 lines) **Purpose:** Performs hour-by-hour energy dispatch simulation for a single BESS + DG configuration.

6.1 Data Structures

SimulationParams (Input)

```
@dataclass
class SimulationParams:
    # Profiles (8,760 values each)
    load_profile: List[float]      # MW per hour
    solar_profile: List[float]     # MW per hour

    # BESS
    bess_capacity: float = 100     # MWh (nameplate)
    bess_charge_power: float = 100 # MW max charge rate
    bess_discharge_power: float = 100 # MW max discharge rate
    bess_efficiency: float = 85    # % round-trip
    bess_min_soc: float = 10       # % minimum SOC
    bess_max_soc: float = 90       # % maximum SOC
    bess_initial_soc: float = 50   # % starting SOC
    bess_daily_cycle_limit: float  # max cycles per day
    bess_enforce_cycle_limit: bool  # hard enforcement flag

    # DG
    dg_enabled: bool = False
    dg_capacity: float = 0         # MW rated capacity
    dg_charges_bess: bool = True   # excess DG power to BESS
    dg_load_priority: str = 'bess_first' # 'bess_first' or 'dg_first'
    dg_takeover_mode: bool = False # DG serves full load when activated

    # Time windows (0-23 hours)
    night_start_hour: int = 18
    night_end_hour: int = 6
    day_start_hour: int = 6
```

```

day_end_hour: int = 18
blackout_start_hour: int = 22
blackout_end_hour: int = 6

# SoC thresholds
dg_soc_on_threshold: float = 30 # % – start DG below this
dg_soc_off_threshold: float = 80 # % – stop DG above this

# Fuel model
dg_fuel_curve_enabled: bool = False
dg_fuel_f0: float = 0.03 # L/hr/kw (no-load)
dg_fuel_f1: float = 0.22 # L/kwh (load)
dg_fuel_flat_rate: float = 0.25 # L/kwh (flat rate)

# Cycle charging
cycle_charging_enabled: bool = False
cycle_charging_min_load_pct: float = 70.0 # %
cycle_charging_off_soc: float = 80.0 # %

```

SimulationState (Mutable During Simulation)

```

@dataclass
class SimulationState:
    # Derived constants (set once at init)
    usable_capacity: float # MWh = capacity × (max_soc - min_soc) / 100
    min_soc_mwh: float # MWh = capacity × min_soc / 100
    max_soc_mwh: float # MWh = capacity × max_soc / 100
    charge_efficiency: float # sqrt(RTE/100)
    discharge_efficiency: float # sqrt(RTE/100)

    # State variables (updated each hour)
    soc: float # Current SOC in MWh
    daily_discharge: float # MWh discharged today
    daily_cycles: float # Equivalent cycles today
    bess_disabled_today: bool # True if cycle limit reached

    # DG tracking
    dg_was_running: bool # DG state in previous hour
    total_dg_starts: int # Cumulative DG start events
    total_dg_runtime_hours: int
    total_dg_fuel_consumed: float # Litres

```

HourlyResult (Output Per Hour)

```

@dataclass
class HourlyResult:
    t: int # Hour index (0-8759)
    day: int # Day of year (1-365)
    hour_of_day: int # Hour (0-23)

    load: float # Load demand (MW)
    solar: float # Solar generation (MW)

    # Energy flow splits
    solar_to_load: float # MW – solar serving load directly
    solar_to_bess: float # MW – solar charging battery
    solar_curtailed: float # MW – solar wasted (BESS full)

    bess_to_load: float # MW – battery discharge to load

    dg_to_load: float # MW – DG serving load
    dg_to_bess: float # MW – DG excess charging BESS
    dg_curtailed: float # MW – DG excess wasted
    dg_running: bool # DG on/off
    dg_mode: str # "OFF", "NORMAL", "TAKEOVER", "CYCLE"
    dg_output_mw: float # Total DG output (MW)
    dg_fuel_consumed: float # Litres this hour

```

```

bess_assisted: bool      # BESS discharged this hour
unserved: float          # MW of unmet demand

soc: float               # SOC after this hour (MWh)
soc_pct: float           # SOC as percentage
daily_cycles: float      # Cumulative cycles today
bess_disabled: bool      # BESS disabled (cycle limit)

bess_state: str          # "Idle", "Charging", "Discharging"
bess_power: float        # Net BESS power (MW, +charge/-discharge)

```

6.2 Initialisation

```

def initialize_simulation(params) -> SimulationState:
    state.usable_capacity = capacity * (max_soc - min_soc) / 100
    state.min_soc_mwh = capacity * min_soc / 100
    state.max_soc_mwh = capacity * max_soc / 100
    state.charge_efficiency = sqrt(bess_efficiency / 100)
    state.discharge_efficiency = sqrt(bess_efficiency / 100)
    state.dg_soc_on_mwh = capacity * dg_soc_on_threshold / 100
    state.dg_soc_off_mwh = capacity * dg_soc_off_threshold / 100
    state.soc = capacity * initial_soc / 100

    # Build 24-element boolean arrays for time windows
    # Night hours (handles midnight wraparound)
    # Day hours
    # Blackout hours

```

6.3 Core Helper Functions

charge_bess()

```

def charge_bess(state, energy_available, charge_power_used):
    """Charge battery. Returns (energy_accepted, new_power_used)."""

    charge_room = max_soc_mwh - soc          # MWh headroom
    charge_power_available = charge_power_limit - charge_power_used # MW remaining

    max_charge = min(
        energy_available,                    # What's offered
        charge_power_available,              # Power limit
        charge_room / charge_efficiency      # Headroom (accounting for losses)
    )

    energy_stored = max_charge * charge_efficiency # Apply charge loss
    soc += energy_stored

    return (max_charge, charge_power_used + max_charge)

```

discharge_bess()

```

def discharge_bess(state, params, energy_needed):
    """Discharge battery. Returns (energy_delivered, discharged_flag)."""

    discharge_available = soc - min_soc_mwh    # MWh above floor

    max_discharge = min(
        energy_needed,                        # What's needed
        discharge_power_limit,                # Power limit
        discharge_available * discharge_efficiency # Available after loss
    )

    energy_withdrawn = max_discharge / discharge_efficiency # More withdrawn than delivered

```

```

soc -= energy_withdrawn

daily_discharge += max_discharge
daily_cycles = daily_discharge / usable_capacity

if enforce_cycle_limit and daily_cycles >= cycle_limit:
    bess_disabled_today = True

return (max_discharge, True)

```

6.4 Efficiency Model

Round-trip efficiency (RTE) is split symmetrically between charge and discharge:

```

RTE = 87% (configurable)
One-way charge efficiency = sqrt(0.87) = 0.9327 (93.27%)
One-way discharge efficiency = sqrt(0.87) = 0.9327 (93.27%)

Charging: energy_stored_in_battery = energy_input × 0.9327
Discharging: energy_delivered = energy_withdrawn_from_battery × 0.9327

Verification: energy_in × 0.9327 × 0.9327 = energy_in × 0.87 = RTE ✓

```

Example: To deliver 10 MWh to load from battery:

- Energy withdrawn from battery = $10 / 0.9327 = 10.72$ MWh
- To store that 10.72 MWh, solar input needed = $10.72 / 0.9327 = 11.49$ MWh
- Round-trip loss = $11.49 - 10 = 1.49$ MWh (13% loss) ✓

6.5 Cycle Counting

Cycles are counted as equivalent full cycles based on energy throughput:

```

daily_discharge += energy_discharged_this_hour # MWh
daily_cycles = daily_discharge / usable_capacity

# Resets at midnight (day boundary)
if current_hour_day != previous_hour_day:
    daily_discharge = 0
    daily_cycles = 0
    bess_disabled_today = False

```

Example: 100 MWh battery, 5–95% SOC → usable_capacity = 90 MWh

- Discharge 45 MWh → $\text{daily_cycles} = 45/90 = 0.5$ cycles
- Discharge another 45 MWh → $\text{daily_cycles} = 90/90 = 1.0$ cycle
- Discharge another 90 MWh → $\text{daily_cycles} = 180/90 = 2.0$ cycles → **BESS disabled**

6.6 Hourly Dispatch Algorithm

For each hour t (0 to 8759):

1. RESET if new day:
 - $\text{daily_discharge} = 0$, $\text{daily_cycles} = 0$, $\text{bess_disabled} = \text{False}$
2. READ inputs:
 - $\text{load} = \text{load_profile}[t]$
 - $\text{solar} = \text{solar_profile}[t]$
3. SOLAR → LOAD (first priority):
 - $\text{solar_to_load} = \min(\text{solar}, \text{load})$

- remaining_load = load - solar_to_load
 - excess_solar = solar - solar_to_load
4. TEMPLATE-SPECIFIC DISPATCH (T0-T6):
Handles: excess solar → BESS, BESS → load, DG activation, DG → BESS
(See Section 6.7 for per-template logic)
5. RECORD unserved:
- unserved = remaining_load (after all sources exhausted)
6. RECORD state:
- soc, soc_pct, daily_cycles, bess_state, dg_mode, fuel
- Determine bess_state: Charging / Discharging / Idle
- Update dg_was_running for next hour's start counting

6.7 Template Dispatch Logic

T0: Solar + BESS Only

```
excess_solar → charge_bess()
remaining_load → discharge_bess()
```

T1: Green Priority (Reactive DG)

With bess_first priority:

```
excess_solar → charge_bess()
remaining_load → discharge_bess()
if remaining_load > 0 → activate_dg()
```

With dg_first priority:

```
excess_solar → charge_bess()
if remaining_load > 0 → activate_dg()
if remaining_load > 0 → discharge_bess()
```

With dg_takeover_mode:

```
Check: can solar + BESS meet full load?
If YES → normal dispatch (no DG)
If NO → DG serves FULL load, ALL solar → charge_bess()
```

T2: DG Night Charge (Proactive)

```
if is_night:
    excess_solar → charge_bess()
    activate_dg() # DG runs proactively at night
    remaining_load → discharge_bess()
else:
    # Same as T1 during day
```

T3: DG Blackout Window

```
if is_blackout:
    # DG cannot run – same as T0
    excess_solar → charge_bess()
    remaining_load → discharge_bess()
else:
    # Same as T1 (DG available)
```

T4: DG Emergency (SoC-Triggered, Anytime)

```

excess_solar → charge_bess()
remaining_load → discharge_bess()
if soc <= soc_on_mwh:
    activate_dg() # Start DG when battery low
if dg_was_running and soc <= soc_off_mwh:
    keep DG running # Hysteresis until SOC recovers

```

T5: DG Day Charge (SoC-Triggered, Day Only)

```

if is_day:
    Same as T4 but DG only available during day hours
else:
    Same as T0 at night

```

T6: DG Night SoC Trigger

```

if is_night:
    Same as T4 but DG only available during night hours
else:
    Same as T0 during day

```

6.8 DG Activation Functions

Standard Activation

```

def activate_dg(state, params, hour, remaining_load, ...):
    dg_output = dg_capacity # Full rated output (binary mode)
    dg_to_load = min(dg_output, remaining_load)
    dg_excess = dg_output - dg_to_load

    if dg_charges_bess and dg_excess > 0:
        dg_to_bess = charge_bess(dg_excess)
        dg_curtailed = dg_excess - dg_to_bess
    else:
        dg_curtailed = dg_excess

    fuel_consumed = calculate_dg_fuel(dg_output)

    if not dg_was_running:
        total_dg_starts += 1
    total_dg_runtime_hours += 1

```

Cycle Charging Activation

```

def activate_dg_cycle_charging(state, params, hour, remaining_load, ...):
    # DG runs at minimum load percentage for efficiency
    min_dg_output = dg_capacity × min_load_pct / 100
    dg_output = max(min_dg_output, remaining_load) # At least min load
    dg_output = min(dg_output, dg_capacity) # Cap at rated

    dg_to_load = min(dg_output, remaining_load)
    dg_excess → all to BESS (that's the purpose of cycle charging)

```

DG Takeover

```

def check_dg_takeover(params, state, hour, ...):
    # Check if green sources can meet full load
    bess_can_provide = min(discharge_power, available_soc × efficiency)
    total_green = solar + bess_can_provide

    if total_green >= load:
        return False # No takeover needed

```

```
# TAKEOVER: DG serves entire load, solar → BESS
solar_to_load = 0
dg_to_load = load # DG output = load exactly
all_solar → charge_bess()
```

6.9 Summary Metrics Calculation

After 8,760 hours, aggregate metrics are computed:

```
# Delivery
hours_full_delivery = count(hours where unserved == 0 and load > 0)
pct_full_delivery = hours_full_delivery / hours_with_load × 100

# Green delivery (delivery without DG)
hours_green_delivery = count(hours where delivered and not dg_running)
pct_green_delivery = hours_green_delivery / hours_with_load × 100

# Energy totals
total_solar_generation = sum(solar)
total_solar_curtailed = sum(solar_curtailed)
pct_solar_curtailed = total_solar_curtailed / total_solar_generation × 100

# Green energy (energy-based, not hour-based)
total_green_energy = sum(solar_to_load + bess_to_load)
total_energy_delivered = sum(solar_to_load + bess_to_load + dg_to_load)
pct_green_energy = total_green_energy / total_energy_delivered × 100

# BESS utilisation
bess_throughput = sum(bess_to_load)
bess_equivalent_cycles = bess_throughput / usable_capacity

# DG
dg_runtime_hours = count(hours where dg_running)
dg_starts = total_dg_starts
total_fuel = total_dg_fuel_consumed
avg_fuel_rate = total_fuel / dg_runtime_hours # L/hr

# Seasonal (March-October, days 60-304)
hours_green_mar_oct = count(green delivery hours in Mar-Oct)
```

7. Step 4 — Results & Analysis

File: pages/Step4_Results.py (881 lines) **Purpose:** Display, filter, compare, and export simulation results from Step 3.

7.1 Results Table

Displays all configurations from the sweep with columns from Section 5.5. Supports:

Filters:

- Show only 100% delivery configurations
- Show only zero-DG configurations
- Sort by: Delivery %, BESS (MWh), Wastage %, Green %, DG Hours

7.2 Configuration Selection

User can select a specific configuration for detailed hourly analysis:

- Container type (radio button)
- BESS capacity (number_input, step=5 MWh)
- DG capacity (number_input, step=5 MW)

7.3 Ranked Recommendations Algorithm

Module: utils/metrics.py — calculate_ranked_recommendations()

4-step filtering and ranking process:

```
STEP 1: Apply delivery requirement
- 'maximize': filter to configs with max delivery %
- 'at_least': filter to configs with delivery >= target_pct
- 'exactly': filter to configs with delivery within ±0.5% of target

STEP 2: Apply secondary constraints
- Remove configs exceeding max_wastage_pct (if set)
- Remove configs exceeding max_dg_hours (if set)

STEP 3: Sort by optimization priority
- 'min_bess_size': ascending BESS capacity
- 'min_wastage': ascending wastage %
- 'min_dg_hours': ascending DG runtime hours
- 'min_cycles': ascending equivalent cycles

STEP 4: Return top 3 recommendations with:
- Config details, all metrics
- Marginal analysis vs next-best config
- Alternative configurations for comparison
```

7.4 Hourly Detail View

For the selected configuration, displays:

- Hourly data table with all energy flows
- Colour coding: Green=Charging, Lavender=Discharging, Yellow=DG, Pink=Unmet
- Date range selector with quick buttons (Week, Month, Summer, Winter)

7.5 Exports

- Hourly data (selected range) → CSV
- Hourly data (full year) → CSV
- Monthly summary → CSV

8. Step 5 — Multi-Year Projection

File: pages/Step5_MultiYear.py (840 lines) **Purpose:** Project BESS performance over 10–20 years accounting for battery degradation.

8.1 Degradation Parameters

Field	Options/Range	Default
Factory degradation	[0, 4, 6, 8, 10] %	0%
Annual degradation	[1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0] %/year	2.5%
Sizing strategy	year1, year10, year20	year1

8.2 Sizing Strategy

Strategy	Description	Nameplate Formula
Year 1 BOL	Size for beginning of life	$\text{target_capacity} = \text{user_selected_capacity}$
Year 10 EOL	Oversize so that Year 10 capacity matches target	$\text{nameplate} = \text{target} / (1 - \text{factory_deg}/100 - \text{annual_deg}/100 \times 10)$
Year 20 EOL	Oversize so that Year 20 capacity matches target	$\text{nameplate} = \text{target} / (1 - \text{factory_deg}/100 - \text{annual_deg}/100 \times 20)$

Container Rounding:

```
actual_nameplate = ceil(raw_nameplate / container_energy) * container_energy
```

8.3 Year-by-Year Capacity Degradation

```
year_1_capacity = nameplate * (1 - factory_degradation / 100)
year_N_capacity = year_1_capacity * (1 - annual_degradation / 100) ^ (N - 1)
```

8.4 Re-simulation at Degraded Capacity

For each year, the full 8,760-hour simulation is re-run with the degraded BESS capacity:

```
for year in range(1, projection_years + 1):
    degraded_capacity = year_N_capacity(year)
    degraded_power = degraded_capacity / duration_hours

    # Re-run dispatch simulation with degraded params
    results = run_simulation(params_with_degraded_capacity)

    # Record annual metrics
    yearly_results.append({
        'Year': year,
        'Capacity_MWh': degraded_capacity,
        'Capacity_%': degraded_capacity / nameplate * 100,
        'Delivery_Hrs': results.hours_full_delivery,
        'Delivery_%': results.pct_full_delivery,
        'DG_Hrs': results.dg_runtime_hours,
        ...
    })
```

8.5 Output Tables

10-Year Annual Projection: Year, Capacity, Capacity %, Delivery Hrs, Load Hrs, Delivery %, DG Hrs, Green Energy MWh, DG Energy MWh, Solar Hrs, BESS Hrs, Curtailed MWh, Wastage %

20-Year Energy Summary: Total solar generated, total DG generated, total curtailed, delivery met, BESS losses, energy balance verification

Monthly Detail: Year × 12 months with monthly breakdown of all metrics

9. Degradation Engine

File: src/degradation_engine.py (528 lines) **Purpose:** Advanced battery degradation modelling using rainflow cycle counting.

9.1 Rainflow Cycle Counting (ASTM E1049-85)

Extracts stress cycles from the SOC history for accurate fatigue life estimation.

Algorithm:

1. Find reversal points (local maxima and minima) in SOC history
2. Apply 4-point algorithm to identify enclosed cycles:
 - Examine consecutive points A, B, C, D
 - If $\text{range}(B,C) \leq \text{range}(A,B)$ and $\text{range}(B,C) \leq \text{range}(C,D)$: extract cycle (B,C)
3. Remaining unmatched points form half-cycles
4. Each cycle has: range_pct (depth), mean_pct (mean SOC), count (1.0 or 0.5)

9.2 Depth-of-Discharge (DoD) Stress Curve

For LFP chemistry, damage is not linear with depth:

DoD (%)	Stress Factor	Meaning
10%	0.3x	Shallow cycling = low damage
20%	0.5x	
40%	0.7x	
60%	0.85x	
80%	1.0x	Baseline
100%	1.2x	Deep cycling = extra damage

9.3 Palmgren-Miner Damage Accumulation

```
# For each extracted cycle:
damage = cycle.count * interpolated_stress_factor(cycle.range_pct)

# Total equivalent full cycles at baseline:
equivalent_full_cycles = sum(damage for all cycles)

# Cycle-induced capacity loss:
cycle_degradation = equivalent_full_cycles * 0.0015 # 0.15% per equiv cycle
```

9.4 Calendar Aging

```
calendar_degradation = years_elapsed × 0.02 # 2% per year
```

9.5 Combined Degradation

```
total_capacity_loss = calendar_degradation + cycle_degradation
remaining_capacity = initial_capacity × (1 - total_capacity_loss)
```

9.6 Strategies

Strategy	Implementation
Standard	Install nameplate, accept degradation
Overbuild	Install nameplate × 1.20, extra capacity absorbs degradation
Augmentation	Install nameplate, add replacement modules at year 8

10. Fuel Model

File: src/fuel_model.py (389 lines) **Purpose:** Calculate diesel generator fuel consumption using the Willans Line model.

10.1 Willans Line Formula

```
fuel_rate (L/hr) = F0 × P_rated_kw + F1 × P_actual_kw
```

Where:

- F_0 = 0.03 L/hr/kW (no-load fuel consumption coefficient)
- F_1 = 0.22 L/kWh (load-dependent fuel consumption coefficient)
- $P_{\text{rated_kw}}$ = DG rated capacity in kW
- $P_{\text{actual_kw}}$ = Actual power output in kW

10.2 Flat Rate Alternative

When fuel curve is disabled:

```
fuel_consumed (L) = P_actual_kw × flat_rate × hours
flat_rate = 0.25 L/kWh (default)
```

10.3 Key Functions

```
calculate_fuel_rate(p_rated_mw, p_actual_mw, f0, f1) → L/hr
calculate_fuel_consumption(p_rated_mw, p_actual_mw, hours, f0, f1) → Litres
calculate_fuel_flat_rate(p_actual_mw, hours, flat_rate) → Litres
calculate_efficiency_at_load(p_rated_mw, load_pct, f0, f1) → {fuel_rate, specific, efficiency}
```

10.4 Worked Examples

25 MW DG at 100% load (Willans):

```
fuel_rate = 0.03 × 25,000 + 0.22 × 25,000 = 750 + 5,500 = 6,250 L/hr
specific = 6,250 / 25,000 = 0.25 L/kWh
```

25 MW DG at 50% load (Willans):

```
fuel_rate = 0.03 × 25,000 + 0.22 × 12,500 = 750 + 2,750 = 3,500 L/hr
specific = 3,500 / 12,500 = 0.28 L/kWh (less efficient at part load)
```

25 MW DG at 100% load (Flat rate):

```
fuel = 25,000 kW × 0.25 L/kWh × 1 hr = 6,250 L
```

11. Step 6 — Green Energy Analysis

File: pages/Step6_Green_Energy_Analysis.py (1,200+ lines) **Purpose:** 4D optimisation sweep across Solar × BESS × Container Type × DG to find configurations meeting green energy targets.

11.1 Optimisation Parameters

Dimension	Min	Max	Step	Unit
Solar capacity	50	200	25	MW
BESS capacity	0	300	25	MWh
Container types	—	—	—	[5mwh_2.5mw, 5mwh_1.25mw]
DG capacity	0	30	5	MW

11.2 Key Metrics

Green Energy Percentage (energy-based):

```
green_energy_pct = (solar_to_load + bess_to_load) / (solar_to_load + bess_to_load + dg_to_load) × 100
```

Green Hours Percentage (hour-based):

```
green_hours_pct = hours_green_delivery / hours_with_load × 100
```

Green Hours March–October (summer season):

```
green_hours_pct_mar_oct = hours_green_mar_oct / hours_with_load_mar_oct × 100
```

11.3 Viability Criteria

A configuration is "viable" when:

```
meets_green_target = green_energy_pct >= green_energy_target_pct # default 50%
meets_wastage_limit = wastage_pct <= max_wastage_pct # default 20%
is_viable = meets_green_target AND meets_wastage_limit
```

11.4 Result Fields

Each configuration in the results includes:

- Solar capacity (MW), BESS capacity (MWh), duration (hr), power (MW), containers (count), DG capacity (MW)
- delivery_pct, green_energy_pct, green_hours_pct, green_hours_pct_mar_oct, wastage_pct
- delivery_hours, load_hours, green_hours, dg_runtime_hours, dg_starts
- solar_generated (GWh), solar_curtailed (GWh), green_delivered (GWh), total_delivered (GWh)
- meets_green_target (bool), meets_wastage_limit (bool), is_viable (bool)

12. Step 7 — Financial Analysis

File: pages/Step7_Financial.py (1,800+ lines) **Engine:** src/financial_model.py (1,005 lines) **Config:** src/financial_config.py (500+ lines) **Purpose:** Calculate ungeared Project IRR using a monthly FCFF chain, replicating off-Grid Solution v8.xlsm.

12.1 FCFF Chain Overview

```
Revenue (FS!row27)
- OPEX (FS!row59)
± NWC (FS!row61)
- CAPEX (FS!rows69:89)
- Tax (D&T!row265)
```

```
= FCFF
```

→ XIRR(FCFF, dates) = Project IRR
→ NPV(FCFF, dates, discount_rate) = Project NPV

Currency: GBP thousands (GBPk) throughout **Granularity:** Monthly periods **Maximum horizon:** 420 months (35 years $\times$ 12)

12.2 FinancialInputs — Complete Parameter Reference

Timing

Parameter	Default	Unit	Description
model_start	2024-07-01	date	Model start date
construction_start	2026-01-01	date	Construction begins
construction_months	18	months	Construction duration
cod_date	2027-07-01	date	Commercial operation date
project_life_years	35	years	Operational life after COD

Solar Generation

Parameter	Default	Unit	Description
solar_capacity_mwp	82.0	MWp	Installed solar capacity

Parameter	Default	Unit	Description
yield_p50	967.0	MWh/MWp/Yr	P50 specific yield
yield_p75	936.0	MWh/MWp/Yr	P75 specific yield
yield_p90	895.0	MWh/MWp/Yr	P90 specific yield
generation_selection	P90	—	Selected yield basis
degradation_pct	0.3	%/yr	Annual PV degradation
seasonality	[1/12] × 12	fraction	Monthly generation shape
outage_selection	0	0/1	Outage enabled
outage_month	1	1–12	Outage month
outage_length_days	14	days	Outage duration

BESS

Parameter	Default	Unit	Description
bess_switch	1	0/1	BESS included
bess_capacity_mw	62.5	MW	BESS power rating
bess_duration_hrs	4.0	hours	BESS duration
bess_operating_life	15	years	BESS operational life
bess_degradation_pct	2.5	%/yr	BESS capacity degradation
bess_merchant_switch	1	0/1	Merchant revenue enabled
bess_scenario	1	int	Merchant scenario selection
bess_merchant_discount	5.0	%	Merchant revenue discount
bess_floor_switch	1	0/1	Floor price contract
bess_floor_price	40.0	GBP/MW/Yr	Floor price
bess_floor_rev_share	10.0	%	Revenue share to optimiser
bess_floor_tenor	10	years	Floor contract tenor

Revenue Streams — PPA

Parameter	Default	Unit	Description
ppa_selection	1	int	PPA scenario
ppa_price_gbp_mwh	50.0	GBP/MWh	PPA strike price
ppa_indexation	CPI	—	Price escalation basis
ppa_flex_pct	0.0	%	PPA flexibility band

Revenue Streams — REGOs

Parameter	Default	Unit	Description
rego_switch	1	0/1	REGOs enabled
rego_price	5.0	GBP/MWh	REGO price
rego_indexation	CPI	—	REGO price escalation
rego_tenor_years	15	years	REGO contract tenor

Revenue Streams — Capacity Market

Parameter	Default	Unit	Description
cm_t1_value	20.0	GBPk/MW/Yr	T-1 auction price
cm_t1_derating	27.15	%	T-1 de-rating factor
cm_t1_tenor	1	years	T-1 contract tenor
cm_t4_value	0.0	GBPk/MW/Yr	T-4 auction price
cm_t4_derating	0.0	%	T-4 de-rating factor
cm_t4_tenor	0	years	T-4 contract tenor

CAPEX (all in GBP/kWp unless noted)

Parameter	Default	Description
capex_epc	400.0	EPC (Engineering, Procurement, Construction)
capex_grid	30.0	Grid connection
capex_development	15.0	Development costs
capex_acquisition	0.0	Acquisition costs
capex_dd	5.0	Due diligence
capex_discharge	0.0	Discharge costs
capex_sdlt	0.0	Stamp Duty Land Tax
capex_land_legal	2.0	Land legal costs
capex_other_finance	0.0	Other finance costs
capex_other_legal	2.0	Other legal costs
capex_land_purchase	0.0	Land purchase
capex_ampyr_tech	0.0	Ampyr technology fee
capex_success_fee	0.0	Success fee

Parameter	Default	Description
capex_community	0.0	Community fund
capex_bess	80.0	BESS capital
capex_landowner_fees	0.0	Landowner fees
capex_insurance	3.0	Insurance during construction
capex_land_lease_constr	0.0	Land lease during construction
capex_asset_adoption	0.0	Asset adoption costs
capex_others	0.0	Other CAPEX
capex_misc	0.0	Miscellaneous
capex_contingency_pct	1.0	Contingency (%)

Solar OPEX (all in GBP/kWp/Year)

Parameter	Default	Description
opex_pv_om	5.48	PV O&M
opex_grid_conn	1.50	Grid connection fee
opex_greenkeeping	0.50	Greenkeeping/landscaping
opex_community	0.00	Community fund
opex_real_estate_tax	1.00	Business rates
opex_non_tech_am	1.00	Non-technical asset management
opex_subsidy_loss	0.00	Subsidy loss
opex_insurance	2.02	Insurance
opex_fixed_lease	0.00	Fixed lease component
opex_corrective_maint	3.20	Corrective maintenance
opex_tech_am	1.50	Technical asset management
opex_social_cost	0.00	Social cost (GBP/MWh, variable)
opex_balancing_cfd	0.00	Balancing/CfD cost (GBP/MWh, variable)

BESS OPEX (all in GBP/MW/Year)

Parameter	Default	Description
bess_opex_om	7.06	O&M

Parameter	Default	Description
bess_opex_import	0.00	Import charges
bess_opex_rates	0.00	Business rates
bess_opex_lease	0.00	Lease costs

Land

Parameter	Default	Unit	Description
fixed_lease_switch	0	0/1	Fixed lease enabled
fixed_lease_acres	200	acres	Lease area
fixed_lease_price	800	GBP/Acre/Yr	Annual rent
rev_dep_lease_switch	0	0/1	Revenue-dependent lease
rev_share_yr1_10	5.0	%	Revenue share years 1–10
rev_share_yr11_35	7.5	%	Revenue share years 11–35
construction_rent_sw	0	0/1	Construction rent enabled
construction_rent	500	GBP/Acre/Yr	Construction period rent

Tax

Parameter	Default	Unit	Description
corp_tax_rate_low	19.0	%	UK small profits rate
corp_tax_rate_high	25.0	%	UK main rate
corp_tax_threshold	250.0	GBPk	Threshold for main rate
taxation_month	12	1–12	Month tax is assessed

Working Capital & Financial

Parameter	Default	Unit	Description
wc_debtors_days	45	days	Receivables collection period
wc_creditors_days	30	days	Payables payment period
project_discount_rate	8.0	%	NPV discount rate
cost_of_capital	6.0	%	WACC
indexation_rate	2.5	%	Annual CPI assumption

12.3 Financial Calculation Functions

build_timeline()

```
total_months = months_between(model_start, cod_date) + project_life_years * 12
dates = [model_start, model_start+1month, ..., end]
is_construction = [construction_start <= date < cod_date]
is_operations = [date >= cod_date]
```

calc_revenue() — Monthly Solar + BESS Revenue (GBPk)

```
for each operations month:
    ops_year = ops_month // 12

    # Solar degradation
    degrad_factor = 1.0 - (degradation_pct / 100) * ops_year # (year 0 = 1.0)

    # Monthly generation (MWh)
    annual_gen = solar_capacity_mwp * selected_yield
    monthly_gen = annual_gen * seasonality[month] * degrad_factor

    # Outage adjustment
    if outage_enabled and matching month:
        monthly_gen *= (1 - outage_days / days_in_month)

    # Price indexation
    indexation_factor = (1 + indexation_rate / 100) ^ ops_year

    # Solar revenue (GBPk)
    solar_rev = monthly_gen * ppa_price * indexation_factor / 1000

    # REGO revenue (GBPk) – added to solar_rev
    if rego_switch and ops_year < rego_tenor:
        solar_rev += monthly_gen * rego_price * indexation_factor / 1000

    # BESS revenue (GBPk)
    if bess_switch and ops_year < bess_operating_life:
        bess_degrad = max(1.0 - bess_degradation_pct/100 * ops_year, 0)
        effective_mw = bess_capacity_mw * bess_degrad

    # Floor revenue
    if bess_floor_switch and ops_year < bess_floor_tenor:
        bess_rev += bess_floor_price * effective_mw / 12 / 1000

    # Capacity Market T-1
    if cm_t1_value > 0 and ops_year < cm_t1_tenor:
        bess_rev += cm_t1_value * (cm_t1_derating/100) * effective_mw / 12

    # Capacity Market T-4
    if cm_t4_value > 0 and ops_year < cm_t4_tenor:
        bess_rev += cm_t4_value * (cm_t4_derating/100) * effective_mw / 12
```

calc_opex() — Monthly OPEX (GBPk, negative values)

```
# Solar fixed OPEX (GBPk/month)
solar_opex_rate = sum of all 11 solar OPEX items # GBP/kWp/Yr
monthly_solar_opex = solar_opex_rate * solar_capacity_mwp / 12

# Variable solar OPEX (GBPk/month)
if social_cost + balancing_cfd > 0:
    variable_opex = monthly_gen * variable_rate * escalation / 1000

# BESS OPEX (GBPk/month)
bess_opex_rate = sum of 4 BESS OPEX items # GBPk/MW/Yr
monthly_bess_opex = bess_opex_rate * bess_capacity_mw / 12

# Land lease
```

```

if fixed_lease:
    land_opex = lease_price × acres / 12 / 1000 # GBPk/month
if rev_dep_lease:
    land_opex = revenue × rev_share_pct

# All escalated annually by indexation_rate
escalation = (1 + indexation_rate / 100) ^ ops_year

```

***calc_capex()* — CAPEX During Construction (GBPk, negative values)**

```

# Sum all 21 CAPEX line items (GBP/kwp)
capex_per_kwp = epc + grid + development + ... + misc

# Total CAPEX (GBPk) = GBP/kwp × Mwp (since kwp/Mwp × GBP/GBPk cancel)
total_capex = capex_per_kwp × solar_capacity_mwp × (1 + contingency_pct / 100)

# Add construction rent if applicable
if construction_rent_sw:
    total_capex += construction_rent × acres × months / 12 / 1000

# Phase evenly over construction months
monthly_capex = total_capex / construction_month_count

```

***calc_depreciation()* — Straight-Line (GBPk, positive values)**

```

ops_months = project_life_years × 12
monthly_depreciation = total_capex / ops_months
# Applied every operations month

```

***calc_ungeared_tax()* — Two-Tier UK Corporation Tax**

```

# Monthly taxable income
monthly_taxable = EBITDA - depreciation

# Accumulate annually, assess at taxation_month
annual_taxable += monthly_taxable each month

at taxation_month:
    # Apply loss carry-forward
    if annual_taxable < 0:
        loss_pool += |annual_taxable|
    else:
        # Use accumulated losses first
        if loss_pool > 0:
            annual_taxable -= min(loss_pool, annual_taxable)
            loss_pool -= used_amount

        # Two-tier tax
        if annual_taxable <= 250 GBPk:
            tax = annual_taxable × 19%
        else:
            tax = 250 × 19% + (annual_taxable - 250) × 25%

        tax_paid[month] = -tax

# Reset annual accumulator

```

***calc_nwc()* — Net Working Capital Change**

```

# NWC balance = Debtors - Creditors
debtors = revenue × 12 × debtors_days / 365 # Annualised receivables
creditors = |opex| × 12 × creditors_days / 365 # Annualised payables

```

```
current_nwc = debtors - creditors

# Change = negative impact on cash flow when NWC increases
nwc_change = -(current_nwc - previous_nwc)
```

FCFF Assembly

```
FCFF = revenue + opex + nwc + capex + tax
# revenue > 0, opex < 0, capex < 0, tax < 0, nwc can be +/-
```

calc_xirr() — XIRR Solver (Newton-Raphson)

```
# XNPV function
XNPV(rate) = sum( cf_i / (1 + rate) ^ ((date_i - date_0).days / 365.25) )

# XNPV derivative
XNPV'(rate) = sum( -frac_i * cf_i / (1 + rate) ^ (frac_i + 1) )

# Newton's method: rate_new = rate - XNPV(rate) / XNPV'(rate)
# Iterate until |rate_new - rate| < 1e-8 or max 200 iterations
# Fallback: bisection search in [-0.99, 10.0] if Newton fails
```

calc_xnpv() — Net Present Value

```
XNPV = sum( cf_i / (1 + rate) ^ ((date_i - date_0).days / 365.25) )
```

12.4 Orchestration: run_financial_model()

```
1. build_timeline()      → dates, is_construction, is_operations
2. calc_capex()          → capex[] (negative during construction)
3. calc_revenue()        → revenue[], solar_rev[], bess_rev[]
4. calc_opex()           → opex[], solar_opex[], bess_opex[]
5. EBITDA = revenue + opex
6. calc_depreciation()   → depreciation[]
7. calc_ungear_tax()     → tax[] (uses EBITDA and depreciation)
8. calc_nwc()            → nwc[] (uses revenue and opex)
9. FCFF = revenue + opex + nwc + capex + tax
10. Cumulative FCFF      → payback month
11. calc_xirr()           → Project IRR
12. calc_xnpv()          → Project NPV
```

12.5 Excel Cell Mapping Reference

The financial model parameters map to specific cells in off-Grid Solution v8.xlsm. The mapping is defined in src/financial_config.py with the structure:

```
INPUT_CELLS = {
    "solar_capacity_mwp": ("Solar&BESS Inputs", 31, "Mwp"),
    "capex_epc": ("Solar&BESS Inputs", 234, "GBP/kWp"),
    "opex_pv_om": ("Solar&BESS Inputs", 215, "GBP/kWp/Yr"),
    "bess_capacity_mw": ("Solar&BESS Inputs", 116, "MW"),
    ...
}
```

Case columns: {1→J, 2→K, 3→L, 4→M, 5→N, 6→O, 7→P, 8→Q} (up to 8 cases).

13. Market Reference Page

File: pages/15_ Market_Reference.py (662 lines) **Purpose:** Reference data for BESS manufacturers, container sizes, and pricing.

13.1 Manufacturer Database (18 Products)

Includes: CATL, BYD, Tesla, Sungrow, Fluence, Envision, Wartsila, Samsung SDI, LG Energy Solution, Northvolt, Saft

Fields per product: Manufacturer, Model, Capacity (MWh), Container Size, Chemistry, Cycle Life, RTE (%), Weight (tonnes), Warranty (years)

13.2 Container Sizes

Category	Configuration	Capacity Range
20-ft Standard	Single rack	2–3 MWh
20-ft Mid-Range	Multi-rack	3–4 MWh
20-ft High-Density	Dense packing	4–5 MWh
20-ft Latest Gen	Advanced cells	5–6 MWh
20-ft Cutting-Edge	Next-gen	6+ MWh
40-ft Integrated	Full system	5–8 MWh
40-ft High-Density	Dense	8–10 MWh

13.3 Duration Classes

Class	Energy:Power Ratio	Use Case
1-hour	1:1	Frequency response
2-hour	1:2	Peak shaving
4-hour	1:4	Load shifting
6-hour	1:6	Extended storage
8-hour	1:8	Long duration

13.4 Pricing Data (2025)

Component	Price
LFP cells	\$40/kWh
Complete BESS (global)	\$125/kWh
Complete BESS (Europe)	\$150–200/kWh

13.5 Cost Calculator

User inputs: Battery Capacity (10–1000 MWh), Price (100–300 \$/kWh), Installation % (10–30%)

$$\text{Total Cost} = \text{Capacity} \times \text{Price} \times (1 + \text{Installation\%})$$

14. Configuration & Defaults Reference

14.1 Global Constants (src/config.py)

Constant	Value	Unit	Description
TARGET_DELIVERY_MW	25.0	MW	Binary delivery target
SOLAR_CAPACITY_MW	67.0	MW	Default solar capacity
MIN_SOC	0.05	fraction	5% minimum SOC
MAX_SOC	0.95	fraction	95% maximum SOC
ROUND_TRIP_EFFICIENCY	0.87	fraction	87% RTE
ONE_WAY_EFFICIENCY	0.9327	fraction	sqrt(0.87)
C_RATE_CHARGE	1.0	C	Charge rate
C_RATE_DISCHARGE	1.0	C	Discharge rate
MIN_BATTERY_SIZE_MWH	10	MWh	Min sizing range
MAX_BATTERY_SIZE_MWH	500	MWh	Max sizing range
BATTERY_SIZE_STEP_MWH	5	MWh	Sizing step
MARGINAL_IMPROVEMENT_THRESHOLD	300	hrs/10MWh	Optimisation knee
HOURS_PER_YEAR	8760	hours	Annual hours
SIMULATION_START_YEAR	2024	year	Timestamp base
DELIVERY_TOLERANCE_MW	0.01	MW	Float tolerance
DEGRADATION_PER_CYCLE	0.0015	fraction	0.15%/cycle
CALENDAR_DEGRADATION_RATE	0.02	fraction	2%/year
DG_CAPACITY_MW	25.0	MW	Default DG size
DG_SOC_ON_THRESHOLD	0.20	fraction	20% SOC trigger
DG_SOC_OFF_THRESHOLD	0.80	fraction	80% SOC off
DG_FUEL_F0	0.03	L/hr/kW	No-load fuel coefficient
DG_FUEL_F1	0.22	L/kWh	Load fuel coefficient
DG_FUEL_FLAT_RATE	0.25	L/kWh	Simple fuel rate

Constant	Value	Unit	Description
DG_FUEL_PRICE_PER_LITER	1.50	\$/L	Fuel price
CYCLE_CHARGING_MIN_LOAD_PCT	70.0	%	Min DG load for cycle charge
CYCLE_CHARGING_OFF_SOC	80.0	%	Stop cycle charging SOC
DEFAULT_BESS_COST_PER_MWH	300,000	\$/MWh	BESS capital cost
DEFAULT_BESS_COST_PER_MW	50,000	\$/MW	BESS power cost
DEFAULT_DISCOUNT_RATE	0.08	fraction	8% WACC
PROJECT_LIFE_YEARS	20	years	Default project life

14.2 DoD Stress Curve

```

DEFAULT_DOD_STRESS_CURVE = {
    10: 0.3, 20: 0.5, 40: 0.7, 60: 0.85, 80: 1.0, 100: 1.2
}

```

15. Appendices

Appendix A: Glossary

Term	Full Name	Description
BESS	Battery Energy Storage System	Grid-scale battery installation
BOL	Beginning of Life	Nameplate capacity at installation
CAPEX	Capital Expenditure	Upfront investment costs
CfD	Contract for Difference	UK renewable energy subsidy scheme
CM	Capacity Market	UK mechanism for ensuring generation adequacy
COD	Commercial Operation Date	Date project begins generating revenue
CPI	Consumer Price Index	Inflation measure for price escalation
DG	Diesel Generator	Backup/supplementary power source
DoD	Depth of Discharge	Percentage of capacity used in a cycle
EBITDA	Earnings Before Interest, Tax, Depreciation & Amortisation	Operating profit
EOL	End of Life	Degraded capacity at specified year
FCFF	Free Cash Flow to Firm	Unlevered cash flow for IRR calculation

Term	Full Name	Description
GBPk	GBP Thousands	Currency unit in financial model
IRR	Internal Rate of Return	Annualised return on investment
LFP	Lithium Iron Phosphate	Battery chemistry type
MWh	Megawatt-hour	Unit of energy
MWp	Megawatt peak	Solar panel rated capacity
NPV	Net Present Value	Present value of future cash flows
NWC	Net Working Capital	Debtors minus creditors
OPEX	Operating Expenditure	Ongoing operational costs
PPA	Power Purchase Agreement	Long-term electricity sales contract
REGO	Renewable Energy Guarantee of Origin	UK green certificate
RTE	Round-Trip Efficiency	Energy out / energy in for storage
SDLT	Stamp Duty Land Tax	UK property transaction tax
SOC	State of Charge	Current battery charge level (%)
XIRR	Extended Internal Rate of Return	IRR with irregular date intervals
XNPV	Extended Net Present Value	NPV with irregular date intervals

Appendix B: File-to-Function Cross-Reference

Function	File	Purpose
build_load_profile()	src/load_builder.py	Generate 8,760-hour load array
load_solar_profile()	src/data_loader.py	Load CSV solar data
scale_solar_profile()	src/data_loader.py	Scale solar to target capacity
infer_template()	src/template_inference.py	Map rules to template T0–T6
init_wizard_state()	src/wizard_state.py	Initialise session state
validate_step_1/2/3/7()	src/wizard_state.py	Validate step inputs
build_simulation_params()	src/wizard_state.py	Convert state to SimulationParams
run_simulation()	src/dispatch_engine.py	Execute 8,760-hour dispatch
charge_bess()	src/dispatch_engine.py	Charge battery with efficiency
discharge_bess()	src/dispatch_engine.py	Discharge battery with efficiency
activate_dg()	src/dispatch_engine.py	Start DG, distribute power

Function	File	Purpose
activate_dg_cycle_charging()	src/dispatch_engine.py	DG at min load, excess to BESS
check_dg_takeover()	src/dispatch_engine.py	DG serves full load if needed
calculate_dg_fuel()	src/dispatch_engine.py	Fuel consumption per hour
dispatch_template_0..6()	src/dispatch_engine.py	Template-specific dispatch logic
calculate_fuel_rate()	src/fuel_model.py	Willans line fuel rate
calculate_fuel_consumption()	src/fuel_model.py	Fuel over time period
extract_cycles()	src/degradation_engine.py	Rainflow cycle extraction
calculate_degradation()	src/degradation_engine.py	Combined calendar + cycle loss
run_green_energy_optimization()	src/green_energy_optimizer.py	4D sweep
calculate_ranked_recommendations()	utils/metrics.py	Filter, rank, recommend configs
find_optimal_battery_size()	utils/metrics.py	Marginal improvement analysis
run_financial_model()	src/financial_model.py	Complete FCFF chain
calc_revenue()	src/financial_model.py	Monthly revenue calculation
calc_opex()	src/financial_model.py	Monthly OPEX calculation
calc_capex()	src/financial_model.py	CAPEX phased over construction
calc_depreciation()	src/financial_model.py	Straight-line depreciation
calc_ungeared_tax()	src/financial_model.py	Two-tier UK corporation tax
calc_nwc()	src/financial_model.py	Net working capital adjustment
calc_xirr()	src/financial_model.py	Newton-Raphson XIRR solver
calc_xnpv()	src/financial_model.py	Extended NPV calculation
inputs_from_wizard_state()	src/financial_model.py	Convert wizard state to inputs

End of document